

استاد: دکتر داوودآبادی

نام درس: مبانی پردازش زبان و گفتار

گزارش: HW2

نام و نام‌خانوادگی: نیوشا یقینی

شماره دانشجویی: ۹۸۵۲۲۳۴۶

فروردین ۱۴۰۴

هر روز هزاران کاربر نظر خود را درباره‌ی محصولات در وب‌سایت ثبت می‌کنند؛ نظراتی که گاه بسیار مثبت‌اند و نشانه‌ی رضایت، و گاه به‌شدت منفی و بیانگر نارضایتی از کیفیت محصول یا خدمات.

سیستمی طراحی کنید که بتواند به‌صورت خودکار احساسات موجود در این نظرات را تشخیص دهد؛ تا تیم‌های مختلف بتوانند به سرعت واکنش نشان دهند، کیفیت خدمات را ارزیابی کنند و حتی تصمیمات استراتژیک‌تری درباره‌ی محصولات بگیرند.

مدلی بسازید که زبان فارسی را درک کند و بتواند احساس هر نظر را به‌درستی تشخیص دهد. در این پروژه، با استفاده از داده‌های واقعی نظرات کاربران دیجی‌کالا، مرحله‌به‌مرحله وارد فرآیند طراحی یک مدل تحلیل احساسات خواهید شد.

از پاکسازی داده‌ها و ساخت واژگان گرفته تا طراحی شبکه‌های عصبی و آموزش آن‌ها.

1 . داده

ابتدا داده‌ها را از آدرس زیر دانلود کردیم.

https://www.kaggle.com/datasets/soheiltehranipour/digikala-comments-persian-sentiment-analysis

ابتدا به دلیل نمایش نادرست داده‌ها با فایل excel داده‌ها را با فرمت "utf-8-sig" ذخیره کردیم تا هم در pandas درست آنها را ببینیم و هم در excel آنها را درست ببینیم.

```
df = pd.read_csv('data.csv', encoding='utf-8')

df.to_csv('new_data.csv', index=False, encoding='utf-8-sig')
```

سپس داده‌ها را خواندیم و اطلاعات اولیه آنها را استخراج کردیم.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3261 entries, 0 to 3260
Data columns (total 3 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   Text        3261 non-null   object
 1   Score       3261 non-null   int64 
 2   Suggestion  3261 non-null   int64 
dtypes: int64(2), object(1)
memory usage: 76.6+ KB
None
             Score  Suggestion
count  3261.000000  3261.00000
mean     74.719411     1.41061
std      21.514015     0.72408
min       0.000000     1.00000
25%      60.000000     1.00000
50%      80.000000     1.00000
75%      92.000000     2.00000
max     100.000000     3.00000
```

با توجه به این نتایج میبینیم که داده null نداریم، همچنین بنظر می‌آید که بازه متغیر score بین ۰ تا ۱۰۰ و بازه متغیر suggestion بین ۱ تا ۳ می‌باشد.

اطلاعات کلی داده‌ها از روی خود وبسایت Kaggle:

Text Raw Comment in Persian	# Score Based on the Description	# Suggestion Based on the Description
3021 unique values	 0 100	 1 3
این اولین تجربه من برای خرید ایفون هست امروز ب دستم رسیده به اسم طلایی هست ولی دراصل رزگلده خیل...	100	1
خرید این محصول رو توصیه میکنم	84	1
ساله این گوشی رو دارم هیچ 1 نقطه ضعفی ازش ندیدم	60	1
سلام خدمت دوستان این گوشی از	96	1

توضیحات داده‌های جدول Suggestion:

مقدار	معنی
1	کاربر خرید محصول را توصیه کرده (نظر مثبت)
2	کاربر خرید را توصیه نکرده (نظر منفی)
3	نظر خنثی دارد
4	محصول را ارزیابی کرده، اما نگفته بخرید یا نه

2. پیش‌پردازش داده‌ها

در این مرحله از ما خواسته شده‌است که با استفاده از کتابخانه Hazm مراحل زیر را روی داده‌ها اعمال کنیم.

- نرمال‌سازی متون فارسی
- حذف علائم نگارشی و نویزهای متنی
- توکن‌سازی جملات به کلمات
- حذف کلمات توقف (Stopwords)
- در صورت نیاز، انجام ریشه‌یابی یا Lemmatization یا Stemming

Normalization:

نرمال‌سازی یعنی اصلاح و استانداردسازی متون برای حذف تفاوت‌های ظاهری که معنی خاصی ندارند. برای مثال:

- تبدیل نیم‌فاصله‌ها به فاصله یا حذف اون‌ها
- تبدیل حروف مختلفی که صدای یکسان دارند (مثل «ک» به «ک» و «ی» به «ی»)
- حذف حروف تکراری و اصلاح مشکلات املایی

کتابخانه Hazm برای نرمال‌سازی داده‌ها یک تابع پیش‌فرض دارد:

normalizer.normalize(text)

این تابع پیش‌فرض فقط **عملیات پایه‌ای نرمال‌سازی** را انجام می‌دهد مثل:

- **تبدیل حروف عربی به فارسی:**
  - ک ⟶ ک
  - ی ⟶ ی
  - ۀ ⟶ ه
- **تبدیل اعداد عربی به فارسی (اگر داده‌ها به این شکل باشند):**
  - ۱۲۳ ⟶ ۱۲۳
- **تبدیل نشانه‌های عربی به فارسی:**
  - مثلا ...به . . .
- **حذف فاصله‌های اضافی (فقط در وسط جمله‌ها و ابتدا و انتها):**
  - تبدیل فاصله‌های تکراری به یک فاصله
  - حذف فاصله‌های اضافی از ابتدا و انتهای جمله‌ها.

چه کارهایی انجام نمی‌دهد؟

- حذف علائم اضافی (فتحه، کسره، ضمه و غیره)
- تبدیل نیم‌فاصله‌ها به فاصله معمولی (\u200c)
- حذف علائم نگارشی مثل ؟ ، ! ، و غیره

مزایا:

- سریع و بدون تغییرات سنگین.
- به مدل اجازه می‌دهد با داده‌ها با شکل طبیعی‌تری کار کند.

معایب:

- اگر داده‌های ورودی دارای **علائم اضافه مثل فتحه، کسره و ضمه** باشند، مدل گیج می‌شود.
- علائم و نیم‌فاصله‌ها ممکن است به عنوان کلمات مجزا شناسایی شوند.
- ممکن است نویز بیشتری به مدل وارد کند.

می‌توان از متغیر persian_style=True در این تابع نیز استفاده کرد، این متغیر ویژگی‌های زیر را نیز اضافه می‌کند:

- **استانداردسازی نیم‌فاصله‌ها (مهم‌ترین تفاوت):** این حالت نیم‌فاصله‌ها رو به شکل صحیح به جمله اضافه می‌کنه.
- **اصلاح علائم نگارشی به سبک فارسی (ویرگول، نقطه ویرگول و ...)**
- **تبدیل اعداد انگلیسی به فارسی**
- **تبدیل حروف انگلیسی بزرگ به کوچک (در صورت وجود)**

ولی برای فعلا ترجیح دادیم اندازه حروف انگلیسی نیز به همان حالت بمانند چون ممکن است اسم خاص باشند و ترجیح میدهیم متفاوت در نظر گرفته شوند.

این متغیر همچنین اعداد انگلیسی رو به فارسی تبدیل میکنه، اما ازونجایی که اعداد ما همگی انگلیسی هستند، صرفا برای ما ککافیه که اعداد همگی از یک جنس باشند.

برای فعلا ساده نگهش میداریم:

```
# Save a copy of original data for comparison
original_texts = df.copy()

# Create a normalizer instance ('persian_style=True' is valid)
normalizer = Normalizer(persian_style=True)
# normalizer = Normalizer()

# Normalization function to apply to each text entry
def preprocess_text(text):
    # Normalize text using hazm
    normalized_text = normalizer.normalize(text)

    return normalized_text

# Apply preprocessing to the 'Text' column
df['Text'] = df['Text'].apply(preprocess_text)
```

نیم فاصله هارا نیز حذف کردیم:

```
Total ZWNJ characters in dataset orginial: 0

After Normalization:
Total ZWNJ characters in dataset before: 7241
Total ZWNJ characters in dataset after: 0
```

تعدادی از تغییراتی که باید اعمال می‌شد را نیز چک کردیم:

```
Before Normalization (Arabic Characters Count): 0
After Normalization (Arabic Characters Count): 0

Before Normalization (Half-Spaces Count): 0
After Normalization (Half-Spaces Count): 2182

Before Normalization (Extra Spaces Count): 2220
After Normalization (Extra Spaces Count): 0
```

Original text digits:

	org_Persian_Digits	org_English_Digits	org_Arabic_Digits
0	0	0	0
1	0	0	0
2	0	1	0
3	0	1	0
4	0	4	0

Normalized text digits:

	norm_Persian_Digits	norm_English_Digits	norm_Arabic_Digits
0	0	0	0
1	0	0	0
2	1	0	0
3	1	0	0
4	4	0	0

Diacritics Count:

محدوده یونی‌کد	توضیح
`\u0617` تا `\u061A`	علامت‌های وقف و مکث در عربی (غیر رایج)
`\u064B` تا `\u0652`	حرکات اصلی عربی و علائم تشدید

```
Before Normalization (Diacritics Count): 99
After Normalization (Diacritics Count): 0
```

نگه نمیدارد:

نوع	مثال‌ها
علائم نگارشی	. ، ؟ ! ( ) [ ] { }
کاراکترهای خاص	* & ^ % $ # @
اموجی‌ها یا سمبل‌ها	™ © ✓ ☺

نگه میدارد:

نوع	بازه Unicode
حروف فارسی	U+0600 – U+06FF
اعداد فارسی	U+06F0 – U+06F9
حروف انگلیسی	U+0041 – U+005A
اعداد انگلیسی	U+0030 – U+0039 یا ۰–۹
فاصله‌ها	U+0020

نرمال‌ساز متن را «قابل‌تحلیل‌تر» می‌کند؛ یعنی کاراکترهایی که قبلاً توی یک حالت خاص یا نامرئی بودند، الان واضح شدند و بنابراین توسط regex تشخیص داده می‌شوند، بنابراین تعداد کاراکترهای ناسازگار هم ممکن است **افزایش پیدا کند**.

```
Original puncs count:  0

Normalized puncs amount:
Total punctuation before: 7241
Total punctuation after: 0
```

Stopword:

کلمات بسیار پرتکراری مثل: «از»، «به»، «در»، «که»، «برای»، «می»، «است» و...
معمولا برای درک معنای اصلی متن مهم نیستند، بنابراین حذفشان باعث کاهش نویز می‌شود.

حال برای حذف stopwords، ابتدا باید لیستی از Stopwordها داشته باشیم. Hazm خودش یک لیست آماده دارد.

برای پروژه‌ای مثل طبقه‌بندی با LSTM/GRU، بهتر است از **Stemmer** استفاده کنیم، چراکه ساده و سریع است و توکن‌های یکنواخت‌تری می‌دهد.

ویژگی	Stemming	Lemmatization
خروجی	ریشه‌ی خام	شکل پایه (گرامری)
دقت	پایین‌تر (ممکنه ناقص یا زیاد ساده‌سازی‌شده باشه)	بالاتر، ولی فقط روی افعال
دامنه پوشش	عمومی برای اکثر کلمات	بیشتر مخصوص افعال
سرعت	خیلی سریع	کندتر
برایLSTM/GRU	مناسب	محدود و ناقص
مثال	«رفته‌ام» ⇦ «رفت»	«رفته‌ام» ⇦ «رفته» ⇦ «رفت (فعل)»

3. آماده‌سازی داده‌ها برای مدل

تعریف واژگان (Vocabulary) از روی توکن‌ها

نگاشت کلمات به اندیس‌های عددی

تبدیل متن‌ها به دنباله‌هایی از اعداد

```
# Flatten the list of token lists into a single list of all tokens
all_tokens = [token for sublist in df['Stemmed_Tokens'] for token in sublist]

# Count the frequency of each token
word_counts = Counter(all_tokens)

sorted_word_counts = word_counts.most_common()

# Build the vocabulary dictionary: each unique word gets a unique index (starting from 2)
# Index 0 and 1 are reserved for special tokens: <PAD> and <UNK>
vocab = {word: idx + 2 for idx, (word, _) in enumerate(word_counts.most_common())}

# Add special tokens to the vocabulary
vocab['<PAD>'] = 0
vocab['<UNK>'] = 1
```

در این قسمت، کد لیست توکن‌های هر سطر (df['Stemmed_Tokens']) را flatten می‌کند. در واقع از یک لیست تو در تو، یک لیست ساده‌ی کلی از همه توکن‌ها می‌سازد. با استفاده از Counter، تعداد تکرار هر توکن در کل داده حساب می‌شود.

word_counts.most_common() لیست کلمات رو بر اساس بیشترین تکرار مرتب می‌کند. با استفاده از enumerate، به هر کلمه یک عدد اختصاص داده می‌شود. ولی از idx+ 2 استفاده می‌کنیم، چون می‌خواهیم عدد ۰ و ۱ را برای توکن‌های خاص رزرو کنیم.

<PAD>: برای padding کردن توالی‌ها

<UNK>: برای کلمات ناشناخته (کلماتی که در واژگان نیستن)

تعدادی از کلمات مرتب شده پرتکرار

```
# Print first 10 items
for word, count in sorted_word_counts[:10]:
    print(f"{word}: {count}")
```

✓ 0.0s

```
: 2142
1331 :خرید
1243 :هس
1215 :کیف
1204 :داره
884 :استفاده
833 :کار
827 :صدا
752 :یه
724 :میشه
```

در واقع در مرحله قبل ما یک دیکشنری به نام vocab می‌سازیم این دیکشنری به ازای هر token یک عدد منحصربه‌فرد به آن اختصاص داده‌است اما مساله اینجاست که برای کدنویسی امن‌تر ما مرحله زیر را هم اضافه می‌کنیم:

۱. ورودی tokens: لیستی از توکن‌ها
۲. Vocab: دیکشنری‌ای که به هر توکن یک **شماره (اندیس)** اختصاص داده
۳. در این تابع:
   - از vocab.get(token, vocab['<UNK>']) استفاده شده‌است.
   - یعنی اگر توکن در واژگان بود ⇦ اندیس آن را برگردان.
   - اگر نبود ⇦ مقدار <UNK> (کلمه ناشناخته) را جایگزین کن.

خروجی این تابع یه لیست از عددهاست که معادل توکن‌ها در واژگان هستند.

برای جلوگیری از خطا در آینده، از vocab[token] مستقیم استفاده نمی‌کنیم. که اگر، دیتافریم تغییر کرد یا توکن‌ها از منبع جدید مثل داده‌های تست یا داده‌های real-time اضافه شدند، به مشکل نخوریم.

یکنواخت‌سازی طول دنباله‌ها با Padding

مدل‌های دنباله‌محور نظیر LSTM و GRU قادر به دریافت ورودی‌هایی با طول‌های متفاوت نیستند. بنابراین، لازم است تمامی دنباله‌ها به طولی یکسان تبدیل شوند. این فرآیند با استفاده از تکنیک **Padding** انجام می‌شود.

در این بخش، از تابع pad_sequence در کتابخانه PyTorch برای یکنواخت‌سازی طول لیست‌ها استفاده کرده‌ایم. عملیات پدینگ با مقدار صفر صورت گرفته که معادل توکن <PAD> (با اندیس ۰) است. اگرچه امکان تعیین مقدار max_len دلخواه برای کنترل طول دنباله‌ها وجود دارد. در اینجا با توجه به اینکه طول پد شده هر دنباله ۷۱۲ است، و میانگین طول واقعی توالی‌ها حدود ۳۱ می‌باشد، بیش از **٪۹۵ محتوای هر سطر** شامل صفر (پدینگ) است. به همین دلیل، افزودن این دنباله‌ها به دیتافریم (DataFrame) می‌تواند باعث افزایش قابل‌توجه حجم حافظه شود.

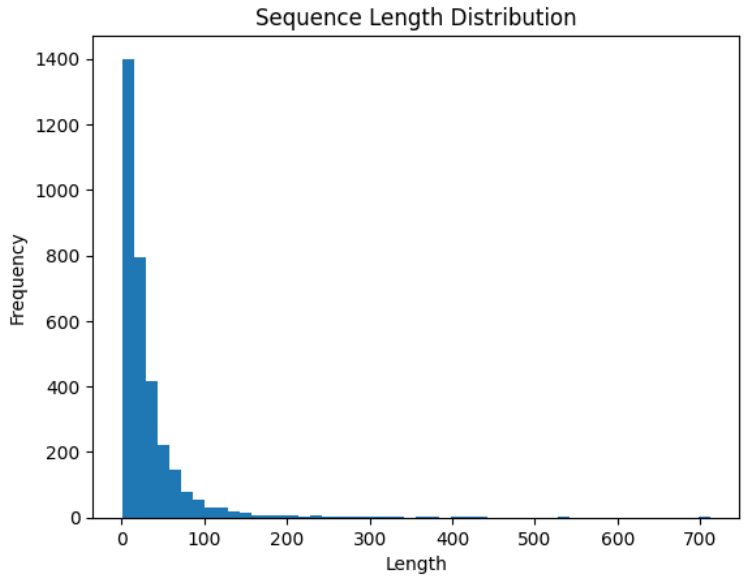
انتخاب طول مناسب برای Padding اهمیت بالایی دارد؛ زیرا می‌تواند هم سرعت آموزش را افزایش دهد و هم مصرف منابع سیستم (RAM و GPU) را کاهش دهد، بدون آن‌که دقت مدل دچار افت شود.

آمار توالی‌ها:

- تعداد نمونه‌ها: ۳۲۶۱
- طول توالی‌ها پس از پدینگ: ۷۱۲
- میانگین طول واقعی: حدود ۳۱

انتخاب مقدار بهینه برای MAX_LEN

برای بهینه‌سازی طول دنباله‌ها، پیشنهاد می‌شود مقدار MAX_LEN به‌گونه‌ای انتخاب شود که **حدود ٪۹۸ دنباله‌ها** را پوشش دهد (و تنها حدود ٪۲ داده بریده شود). این رویکرد تعادلی مناسب میان حفظ داده‌ها و مدیریت منابع فراهم می‌کند. در عمل، مقادیر MAX_LEN بین **۱۰۰ تا ۲۰۰** برای متن‌های کوتاه (مانند نظرات کاربران) کاملاً کافی و رایج است.



پیاده‌سازی کلاس Dataset سفارشی در Pytorch

```python
import torch
from torch.utils.data import Dataset

class TextDataset(Dataset):
    def __init__(self, sequences, labels):
        self.sequences = sequences          # tensor of padded input sequences
        self.labels = torch.tensor(labels)  # tensor of labels

    def __len__(self):
        return len(self.sequences)

    def __getitem__(self, idx):
        return self.sequences[idx], self.labels[idx]
```

برای استفاده ساختاریافته از داده‌ها در فرآیند آموزش مدل در PyTorch، نیاز است داده‌ها در قالب مشخصی قرار گیرند تا توسط ماژول DataLoader قابل پردازش باشند DataLoader وظایف زیر را برعهده دارد:

- تقسیم داده‌ها به دسته‌های کوچک‌تر (Batching)
- انجام ترتیب‌زدایی تصادفی (Shuffling)، در صورت نیاز
- تغذیه خودکار داده‌ها به مدل در هر Epoch

اما برای استفاده از DataLoader، باید داده‌ها را از طریق یک کلاس خاص در اختیار آن قرار داد، که این کلاس با نام **Dataset** شناخته می‌شود.

حال برای ساخت Dataset سفارشی در PyTorch، یک کلاس جدید تعریف کرده و متدهای زیر را در آن پیاده‌سازی می‌کنیم:

- __len__: تعداد کل نمونه‌ها را مشخص می‌کند.
- __getitem__: نحوه‌ی بازیابی یک نمونه (داده + برچسب) را مشخص می‌کند.

این ساختار باعث می‌شود بتوان داده‌ها را به شکل مؤثری برای آموزش آماده کرد و به مدل ارائه داد.

اهمیت انتخاب لیبل مناسب

انتخاب ستون مناسب برای برچسب (Label) نقش مهمی در تعیین نتایج دارد و با توجه به معیارهایی مثل نوع مسئله (دسته‌بندی دودویی، چندکلاسه، یا رگرسیون)، نوع تابع زیان مورد استفاده (Loss Function) و همچنین معیارهای ارزیابی عملکرد مدل، انتخاب می‌شود.

با توجه به ساختار پروژه‌ی حاضر که از مدل LSTM همراه با تابع زیان CrossEntropyLoss استفاده می‌کند، ستون **Suggestion** گزینه‌ی مناسبی برای برچسب‌دهی است.

دلایل این انتخاب عبارت‌اند از:

۱. **نوع داده**: این ستون شامل اعداد صحیح است که معرف دسته‌بندی گسسته هستند.
۲. **سازگاری با تابع زیان**: خروجی این ستون مستقیماً قابل استفاده در CrossEntropyLoss است، که ویژه‌ی مسائل طبقه‌بندی طراحی شده‌است.
۳. **انعطاف‌پذیری در کاربرد**: بسته به نوع داده می‌توان از این لیبل هم در مدل‌های دودویی و هم در مدل‌های چندکلاسه استفاده کرد.

استفاده از DataLoader برای ساخت Batchهای آموزشی

```
from torch.utils.data import DataLoader

batch_size = 32

train_loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
```

در فرآیند آموزش مدل‌های یادگیری عمیق، استفاده از Batch در هر مرحله آموزش (Iteration) نقش مهمی در بهره‌وری و مدیریت منابع دارد. **Batch size** مشخص می‌کند که در هر مرحله، چند نمونه از داده به مدل داده شود.

در این پروژه، با وجود حدود ۳۲۰۰ نمونه داده، مقدار batch_size برابر با **۳۲** در نظر گرفته شده است. این انتخاب بر اساس ملاحظات زیر انجام شده است:

- حجم داده‌ها نسبتاً کوچک است.
- پروژه در مرحله توسعه و آزمایش قرار دارد.
- مصرف حافظه با این اندازه کنترل‌پذیر باقی می‌ماند.
- این مقدار به‌خوبی با هر دو محیط CPU و GPU سازگار است و عملکرد قابل قبولی دارد.

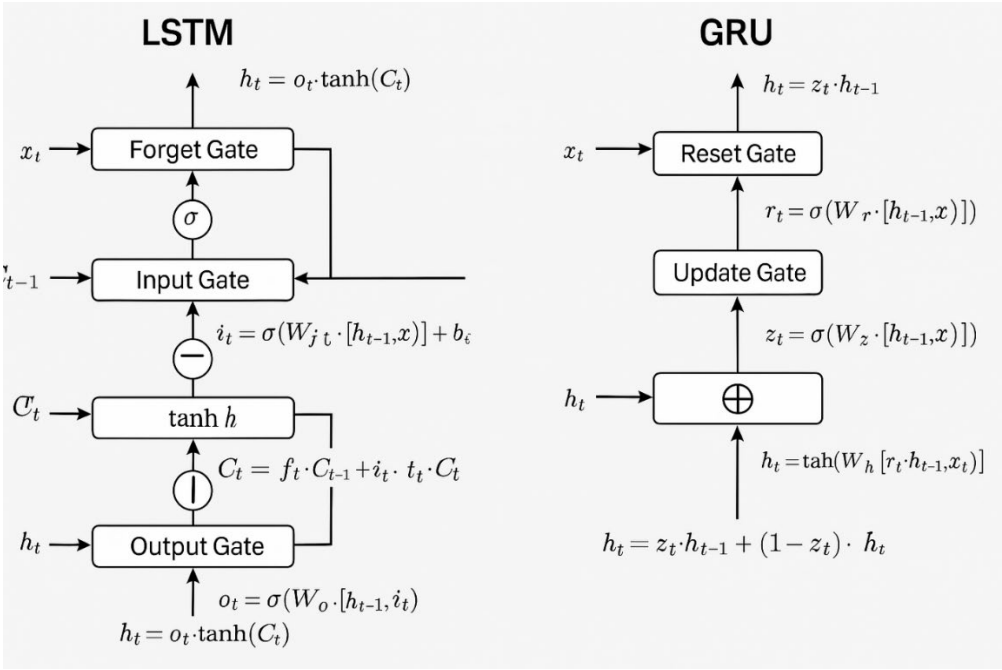
استفاده از مقدار مناسب برای Batch size می‌تواند تعادلی بین دقت مدل، سرعت آموزش، و مصرف منابع سخت‌افزاری ایجاد کند.

4. تعریف مدل در pytorch

یک مدل ساده طبقه‌بندی متن با استفاده از ساختار زیر طراحی کنید:

- لایه **Embedding** برای تبدیل اندیس‌ها به بردارهای تعبیه
- شبکه بازگشتی **LSTM یا GRU** برای تحلیل توالی کلمات
- لایه **Fully Connected** برای پیش‌بینی برچسب متن

تفاوت LSTM و GRU:




```python
class SentimentLSTM(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim, pad_idx):
        super(SentimentLSTM, self).__init__()

        # 1. Embedding Layer
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=pad_idx)

        # 2. LSTM Layer (can replace with GRU if needed)
        self.lstm = nn.LSTM(embedding_dim, hidden_dim, batch_first=True)

        # 3. Dropout + Fully Connected Layer
        self.dropout = nn.Dropout(0.3)
        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        # x shape: [batch_size, sequence_length]
        embedded = self.embedding(x)                    # [batch_size, seq_len, embedding_dim]
        lstm_out, (hidden, cell) = self.lstm(embedded)  # hidden: [1, batch_size, hidden_dim]
        output = self.dropout(hidden[-1])               # get last hidden state
        output = self.fc(output)                        # [batch_size, output_dim]
        return output
```

```python
vocab_size = len(vocab)
embedding_dim = 100
hidden_dim = padded_sequences.shape[1]
# hidden_dim = 123 # the size of the padding vector
output_dim = len(label_map)
# output_dim = 3 # size of different labels
pad_idx = vocab['<PAD>']

model = SentimentLSTM(vocab_size, embedding_dim, hidden_dim, output_dim, pad_idx)
```

با توجه به نیاز مساله مدل آموزشی مربوطه را طراحی کردیم، که ورودی آن توالی‌ای از توکن‌هاست (نماینده‌ی جملات یا نظرات کاربران) و خروجی‌اش یک پیش‌بینی از کلاس احساسات (مثلاً مثبت، منفی، خنثی).

- ابتدا اجزای کلاس را با تابع __init__ تعریف می‌کنیم.
- (Embedding Layer):
این بخش برای تبدیل اندیس توکن‌ها به بردارهایی با ابعاد ثابت استفاده می‌شود.کار این لایه:
  - فراهم‌کردن ورودی پیوسته و قابل یادگیری برای مدل
  - امکان نادیده گرفتن توکن‌های padding از طریق padding_idx
- (LSTM Layer):
لایه‌ی مرکزی مدل که توالی embedding را دریافت کرده و رابطه‌های زمانی بین توکن‌ها را استخراج می‌کند. کار این لایه:
  - ثبت وابستگی‌های معنایی و ساختاری در متن
  - خروجی گرفتن از آخرین وضعیت مخفی به‌عنوان خلاصه‌ی کل جمله

- لایه Dropout و Fully Connected:
پس از LSTM، برای کاهش overfitting از dropout استفاده شده، سپس خروجی وارد یک لایه خطی می‌شود که آن را به تعداد کلاس‌ها نگاشت می‌کند. کار این لایه:
  - حذف تصادفی بخشی از ویژگی‌ها در مرحله‌ی آموزش (برای افزایش تعمیم‌پذیری)
  - تولید خروجی نهایی مدل برای طبقه‌بندی

5. آموزش مدل

```
import torch.nn as nn

criterion = nn.CrossEntropyLoss()

import torch.optim as optim

optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
model.to(device)

print(next(model.parameters()).device)  # cuda:0
```

در این بخش، مدل برای آموزش روی داده‌ها آماده‌سازی شده است. ابتدا با دستور model.to(device) مدل به دستگاه محاسباتی GPU منتقل می‌شود. سپس تابع زیان CrossEntropyLoss برای مسائل طبقه‌بندی چندکلاسه تعریف شده و بهینه‌سازی پارامترهای مدل با الگوریتم Adam انجام می‌گیرد.

```python
for epoch in range(num_epochs):
    model.train()  # set the model to training mode
    total_loss = 0

    for batch in train_loader:
        inputs, labels = batch  # batch contains [batch_size, seq_len], [batch_size]

        # for using gpu
        inputs = inputs.to(device)
        labels = labels.to(device)

        # Forward pass: compute model predictions
        outputs = model(inputs)  # shape: [batch_size, num_classes]

        # Compute loss
        loss = criterion(outputs, labels)

        # Zero out gradients from previous step
        optimizer.zero_grad()

        # Backward pass: compute gradients
        loss.backward()

        # Update model weights
        optimizer.step()

        # Accumulate batch loss
        total_loss += loss.item()

    # Compute and print average loss for the epoch
    avg_loss = total_loss / len(train_loader)
    print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {avg_loss:.4f}")
```

در حلقه‌ی آموزش، برای هر epoch:

- مدل در وضعیت آموزش قرار می‌گیرد.
- ورودی‌ها و برچسب‌ها به دستگاه منتقل می‌شوند.
- خروجی مدل محاسبه می‌شود.
- مقدار زیان با استفاده از خروجی مدل و برچسب واقعی محاسبه می‌گردد.
- گرادیان‌ها پاک‌سازی می‌شوند.
- عملیات پخش به عقب (backward) برای محاسبه‌ی گرادیان‌ها اجرا می‌شود.
- پارامترهای مدل به‌روزرسانی می‌شوند.
- مقدار زیان هر batch برای محاسبه‌ی میانگین زیان ذخیره می‌گردد.

در پایان هرepoch ، میانگین زیان محاسبه و گزارش می‌شود. (Test Loss)

```
Epoch [1/50], Loss: 0.7548
Epoch [2/50], Loss: 0.7547
Epoch [3/50], Loss: 0.7526
Epoch [4/50], Loss: 0.7506
Epoch [5/50], Loss: 0.7525
Epoch [6/50], Loss: 0.7501
Epoch [7/50], Loss: 0.7486
Epoch [8/50], Loss: 0.7453
Epoch [9/50], Loss: 0.7455
Epoch [10/50], Loss: 0.7466
Epoch [11/50], Loss: 0.7401
Epoch [12/50], Loss: 0.7342
Epoch [13/50], Loss: 0.7324
Epoch [14/50], Loss: 0.7493
Epoch [15/50], Loss: 0.7495
Epoch [16/50], Loss: 0.7297
Epoch [17/50], Loss: 0.7275
Epoch [18/50], Loss: 0.7224
Epoch [19/50], Loss: 0.7165
Epoch [20/50], Loss: 0.7138
Epoch [21/50], Loss: 0.7093
Epoch [22/50], Loss: 0.7114
Epoch [23/50], Loss: 0.7023
Epoch [24/50], Loss: 0.6984
Epoch [25/50], Loss: 0.6937
...
Epoch [47/50], Loss: 0.5247
Epoch [48/50], Loss: 0.5144
Epoch [49/50], Loss: 0.5063
Epoch [50/50], Loss: 0.5048
```

6. ارزیابی مدل

کدهای این بخش به دلیل طولانی بودن در گزارش آورده نشده‌اند، اینجا توضیح آنها را قرار می‌دهیم، برای خود کدها به پیاده سازی مربوطه مراجعه شود.

در این بخش، فرآیند کامل آموزش، ارزیابی، ذخیره‌سازی بهترین مدل، و پیش‌بینی نمونه‌های جدید پیاده‌سازی شده است.

ابتدا داده‌ها به دو بخش آموزش و اعتبارسنجی تقسیم شده‌اند. در این تقسیم‌بندی از train_test_split با نسبت ۲۰/۸۰ استفاده شده و با پارامتر stratify توزیع برچسب‌ها در دو مجموعه حفظ شده است. داده‌های تقسیم‌شده به شکل شیءهای Dataset در آمده‌اند و سپس با DataLoader بسته‌بندی شده‌اند تا در حین آموزش، batchهای مشخص‌شده (اینجا ۳۲تایی) تولید شود.

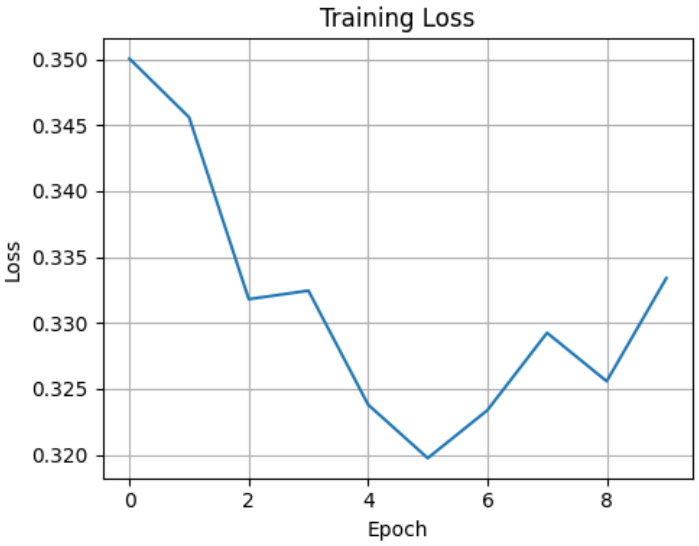
حلقه‌ی اصلی آموزش مدل به‌مدت ۱۰ دوره (epoch) اجرا می‌شود. در هر دوره:

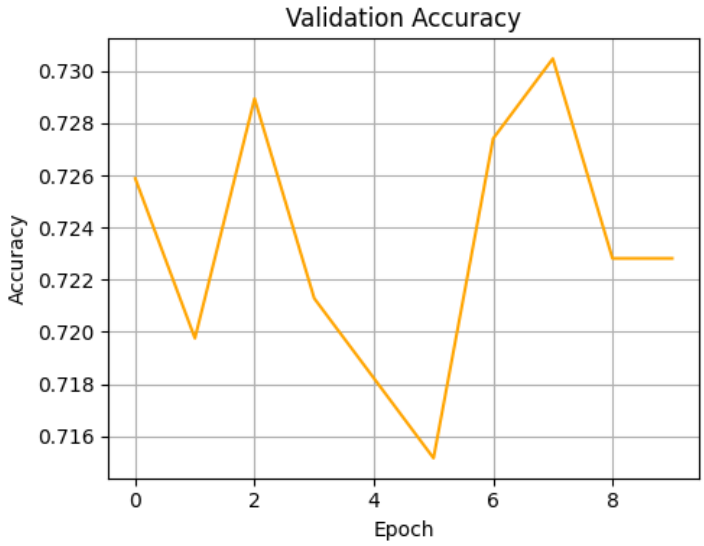
- مدل در وضعیت train قرار می‌گیرد.
- batchهای آموزشی بارگذاری می‌شوند و به GPU منتقل می‌گردند.
- پیش‌بینی مدل انجام می‌شود و زیان مربوطه محاسبه می‌گردد.
- گرادیان‌ها محاسبه شده و پارامترها به‌روزرسانی می‌شوند.
- مجموع زیان‌ها برای گزارش‌گیری در پایان هر دوره ذخیره می‌شود.

پس از اتمام هر دوره، مدل وارد حالت eval می‌شود و با استفاده از داده‌های اعتبارسنجی، بدون محاسبه‌ی گرادیان (با torch.no_grad())، پیش‌بینی انجام می‌گیرد. نتایج خروجی با برچسب‌های واقعی مقایسه شده و چهار معیار استاندارد — دقت (Accuracy)، دقت طبقه‌بندی (Precision)، بازخوانی (Recall) و میانگین — F1 محاسبه می‌شوند.اگر مقدار F1 بهتری نسبت به مدل‌های قبلی به‌دست آمده باشد، مدل فعلی به‌عنوان بهترین مدل ذخیره می‌شود. همچنین، مقدار معیارها و شماره‌ی دوره در متغیرهایی ذخیره می‌شوند تا در پایان گزارش داده شوند.

در انتهای آموزش، با استفاده از matplotlib نمودارهایی برای تغییرات زیان آموزش و دقت اعتبارسنجی در طول دوره‌ها ترسیم می‌شود. و بهترین مدل بارگذاری می‌شود و پنج نمونه از داده‌های مجموعه اعتبارسنجی به مدل داده می‌شود. خروجی پیش‌بینی‌شده‌ی مدل در مقایسه با برچسب واقعی برای هر نمونه چاپ می‌گردد.

```
Epoch [1/10], Train Loss: 0.3500
Validation Accuracy: 0.7259
Precision: 0.8169 | Recall: 0.7259 | F1 Score: 0.7535
------------------------------------------------------------
Best model saved at epoch 1 with F1 Score: 0.7535
Epoch [2/10], Train Loss: 0.3456
Validation Accuracy: 0.7198
Precision: 0.8038 | Recall: 0.7198 | F1 Score: 0.7444
------------------------------------------------------------
Epoch [3/10], Train Loss: 0.3318
Validation Accuracy: 0.7289
Precision: 0.8032 | Recall: 0.7289 | F1 Score: 0.7508
------------------------------------------------------------
Epoch [4/10], Train Loss: 0.3325
Validation Accuracy: 0.7213
Precision: 0.7950 | Recall: 0.7213 | F1 Score: 0.7459
------------------------------------------------------------
Epoch [5/10], Train Loss: 0.3238
Validation Accuracy: 0.7182
Precision: 0.7973 | Recall: 0.7182 | F1 Score: 0.7425
------------------------------------------------------------
Epoch [6/10], Train Loss: 0.3198
Validation Accuracy: 0.7152
Precision: 0.8026 | Recall: 0.7152 | F1 Score: 0.7408
------------------------------------------------------------
...
Epoch [10/10], Train Loss: 0.3334
Validation Accuracy: 0.7228
Precision: 0.8072 | Recall: 0.7228 | F1 Score: 0.7506
------------------------------------------------------------
```






```
Best Validation Results:
Epoch: 8
Accuracy : 0.7305
Precision: 0.8069
Recall   : 0.7305
F1 Score : 0.7538

Sample Predictions from Validation Set:
Sample 1: True Label = 0 | Predicted = 2
Sample 2: True Label = 0 | Predicted = 0
Sample 3: True Label = 0 | Predicted = 0
Sample 4: True Label = 0 | Predicted = 0
Sample 5: True Label = 0 | Predicted = 0
```